

---

# MIT805: Big data, Assignment: Part 2

**GitHub Link:** <https://github.com/potgieterphiline/MIT805-BigData-Project-Part2>

**Tshwanelo Mathala | u21588997**

**Philine Potgieter | u27043275**

---

## 1 Processing objective

The question that this assignment will answer is:

*Which night-time routes or zones have the lowest average travel speed, and when does this occur?*

This question requires the trip duration and the average speed to be calculated per pick up period, pick up date, pick up hour, pick up day, start zone, end zone, route, trip duration and speed. To prepare the data initial steps were put in place to cater for impossible durations, distances and speeds.

The outcome will be to identify night-time routes and zones that consistently experience the lowest average travel speeds and determine the hour of day and periods when these conditions occur. The analysis is expected to show spatial and temporal traffic patterns that may indicate congestion, roadworks, infrastructure constraints or operational inefficiencies. This will enable better planning for the organisation as well as enhance passenger experience in knowing estimated travelling times.

## 2 Data preprocessing

The preprocessing was used to clean the NYC Yellow Taxi data and remove records that did not make sense. Several checks were performed:

- Trips shorter than 10 seconds were removed as they were considered unrealistic.
- Trips with a distance of 0 or less were removed.
- Trips with an average speed above 100 mph were removed because this was considered unrealistic.
- The 99th percentile was used to check for unusually long trip distances and durations. These were reviewed as possible outliers but were not removed.

These checks helped make the dataset cleaner and more reliable for further analysis and modelling

## 3 Mapping / Transformation

Raw trip records were filtered to 2024 and 2025 data and transformed by deriving route identifiers, pickup period, pickup hour, day of month and trip duration. Route identifiers were created by concatenating pickup and drop-off location IDs. Trip durations were calculated as the difference between pickup and drop-off times in seconds. This transformed the data into a structured format for route-level traffic analysis (Apache Spark, 2025).

Below are the top rows of the transformed dataset:

| timepoint_pickup_datetime | start_zone | end_zone | trip_hour | trip_day | route   | trip_duration_seconds | trip_distance | period |
|---------------------------|------------|----------|-----------|----------|---------|-----------------------|---------------|--------|
| 2025-05-01 00:07:06       | 140        | 202      | 0         | 1        | 140-202 | 1029                  | 3.7           | 202505 |
| 2025-05-01 00:07:44       | 234        | 161      | 0         | 1        | 234-161 | 403                   | 1.03          | 202505 |
| 2025-05-01 00:15:56       | 161        | 234      | 0         | 1        | 161-234 | 477                   | 1.57          | 202505 |
| 2025-05-01 00:00:09       | 138        | 90       | 0         | 1        | 138-90  | 1520                  | 9.48          | 202505 |
| 2025-05-01 00:45:07       | 90         | 231      | 0         | 1        | 90-231  | 458                   | 1.8           | 202505 |

## 4 Shuffle / Grouping

During the shuffle phase, Spark redistributed records across worker nodes based on the grouping criteria which includes route, pickup zone, pickup hour and pickup day and trip period (Apache Spark, 2025). Trips sharing the same route-time combination were assembled on the same partition to allow aggregation calculations to be performed. The distributed grouping process enables efficiencies within big data processing and enables rapid response to urgent business questions.

Below are the top rows of the grouped dataset:

| route   | start_zone | trip_hour | trip_day | period | total_trip_distance | total_trip_duration_seconds | trip_count |
|---------|------------|-----------|----------|--------|---------------------|-----------------------------|------------|
| 246-161 | 246        | 0         | 1        | 202505 | 2.5                 | 651                         | 1          |
| 87-85   | 87         | 0         | 1        | 202505 | 12.74               | 3278                        | 2          |
| 230-164 | 230        | 0         | 1        | 202505 | 5.63                | 2058                        | 5          |
| 164-114 | 164        | 0         | 1        | 202505 | 4.95                | 1499                        | 3          |
| 246-87  | 246        | 0         | 1        | 202505 | 4.91                | 846                         | 1          |

## 5 Reduction / Aggregation

Total trip distance, total trip duration and trip count were aggregated for each route-time combination. Average route speed was then calculated by dividing the aggregated total distance travelled by total travel time. This action reduced a large dataset, consisting of many records, into a summarized version with route-level performance indicators (Apache Spark, 2025).

## 6 Spark execution & DAG

The parsed and analysed logical plans show the sequence of transformations applied to the dataset. This includes filtering the records to the 2024 and 2025 periods, deriving the various feature indicators, and calculating the trip durations. Spark combines these transformations into a DAG, enabling processing logic to be optimised before execution (Apache Spark, 2025; Zaharia et al., 2012).

The optimised logical plan shows Spark's ability to reduce unnecessary processing by keeping only the columns required. This reduces data movement and improves execution efficiency (Apache Spark, 2025). The physical plan gives evidence of distributed execution across the Spark cluster.

The HashAggregate operations perform the aggregation of the trip distance and trip duration for each route, zone, hour, day and period combinations. The Exchange HashPartitioning operation indicates a shuffle phase, here records are distributed across partitions so that all records allocated to the same grouping key are processed together (Apache Spark, 2025). The filtering and feature engineering steps were performed in the first Spark stage. The groupby operation then triggered a shuffle, where records were redistributed across partitions. A second stage performed the final aggregation calculations on the

grouped data. Each partition in the hash partitioning process is processed by individual Spark tasks running in parallel across worker nodes (Apache Spark, 2025).

Compared to traditional Hadoop MapReduce, Spark has proven to give better performance as the intermediate results are retained in memory rather than repeatedly writing to disk between map and reduce phases. Spark's DAG-based execution model also allows for multiple transformations to be optimised into a single execution plan. With Hadoop MapReduce, each processing stage is executed as a separate job (Zaharia et al., 2012; Dean & Ghemawat, 2008).

A copy of the Spark DAG execution plan can be found in the Appendix section of the assignment report.

## **7 Results interpretation**

After cleaning, 25,869,746 night-time trips from 2024 to 2025 were aggregated. Average speed was calculated as total miles divided by total seconds, multiplied by 3600, so that each trip is weighted by its length

The processing shows that speed changes a lot through the night (Figure 1). Speed is lowest at 20:00, at 13.0 mph, and highest at 04:00, at 19.6 mph. Although it dips slightly at 01:00 and 02:00, the trend is upward. Furthermore, the slowest hours are also the busiest. Each hour from 20:00 to 22:00 has around 5 million trips, while 04:00 has only about 0.58 million. This suggests that heavy traffic is a main reason for the slow speeds.

The processing also shows where the slowest speeds are (Figure 2). All ten of the slowest pickup zones are in central Manhattan, with speeds between 10.3 and 10.6 mph. Figure 3 then combines place and time. The lowest speed is in West Village at 20:00, at 8.8 mph. Greenwich Village South and Penn Station/Madison Sq West follow at 8.9 mph. Most zones speed up as the night goes on. For example, Midtown Center and Midtown East reach 26.1 mph by 04:00. However, the Village zones only reach about 15.6 to 15.9 mph at 04:00, so they stay slow for longer. The two airports are different, with speeds above 23 mph at every hour.

Additionally, speeds follow the seasons (Figure 4). In both years, speed is highest in August and lowest in December. Speeds in 2025 are also higher than in 2024 from March to December. Although congestion pricing started in January 2025, this analysis cannot show that it caused the difference, because weather, events and other factors were not checked.

In summary, the results answer the question directly. The lowest night-time speeds are in central Manhattan, mainly in the Village and Penn Station areas, and they happen between 20:00 and 22:00. These zones and hours show where further study would be most useful.

## **8 Results visualization**

Average speed per hour

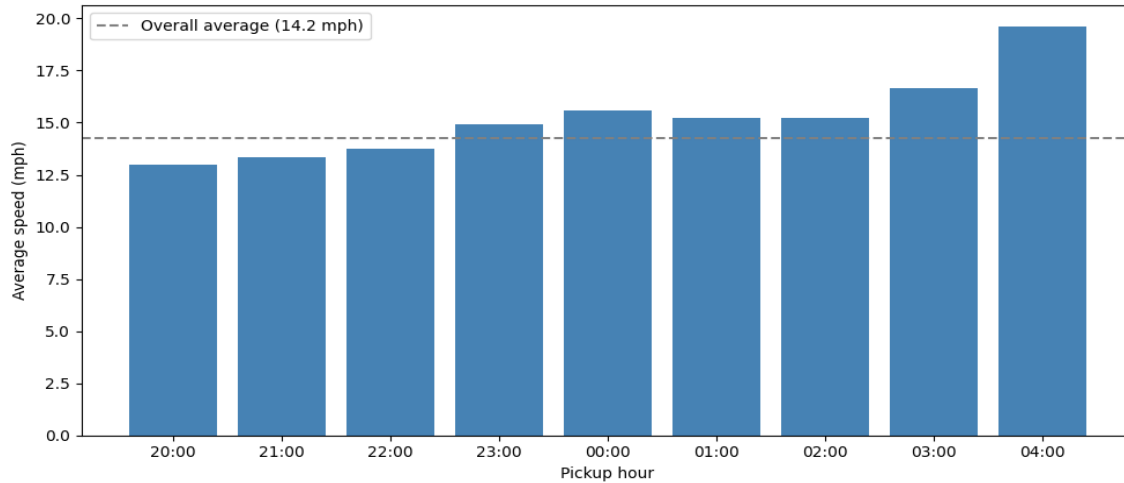


Figure 1: Average taxi speed by pickup hour (night-time, 2024-2025)

This bar chart shows when speeds are lowest. Speed is about 13.0 mph at 20:00 and goes up to 19.6 mph at 04:00. The bars start at zero, so the differences are not made to look bigger than they are. The dashed line shows the overall average of 14.2 mph.

#### Slowest zones

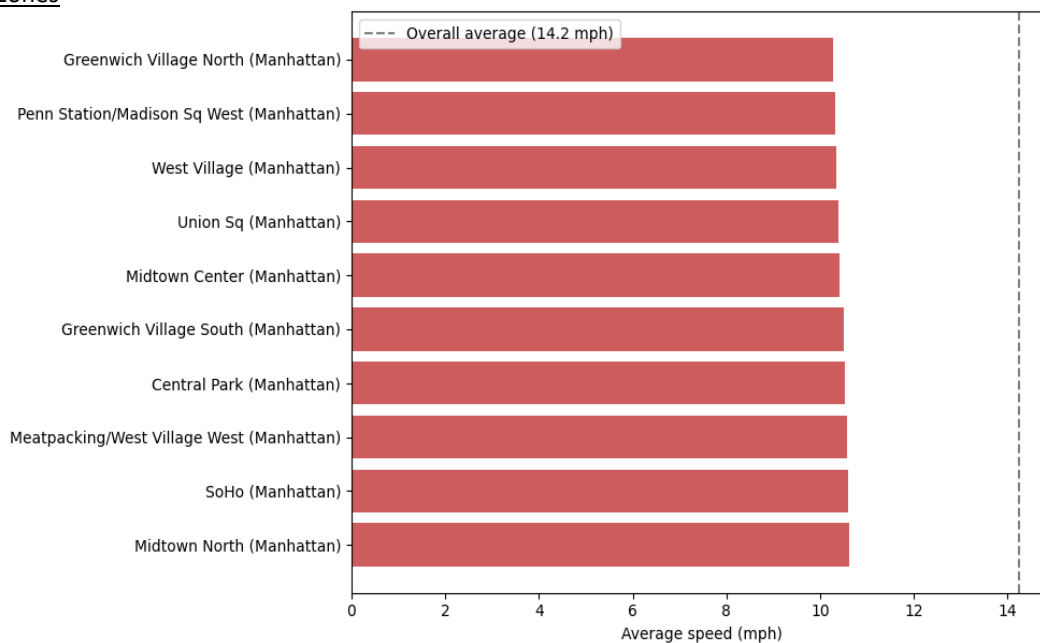


Figure 2: The 10 slowest night-time pickup zones (2024-2025)

This bar chart shows where speeds are lowest. All ten zones are in Manhattan, and all are well below the overall average of 14.2 mph. Zone names are used instead of ID numbers, so the results are easy to read. We also left out zones with fewer than 1,000 trips, because their averages are not reliable.

#### Average speed by zone at each hour

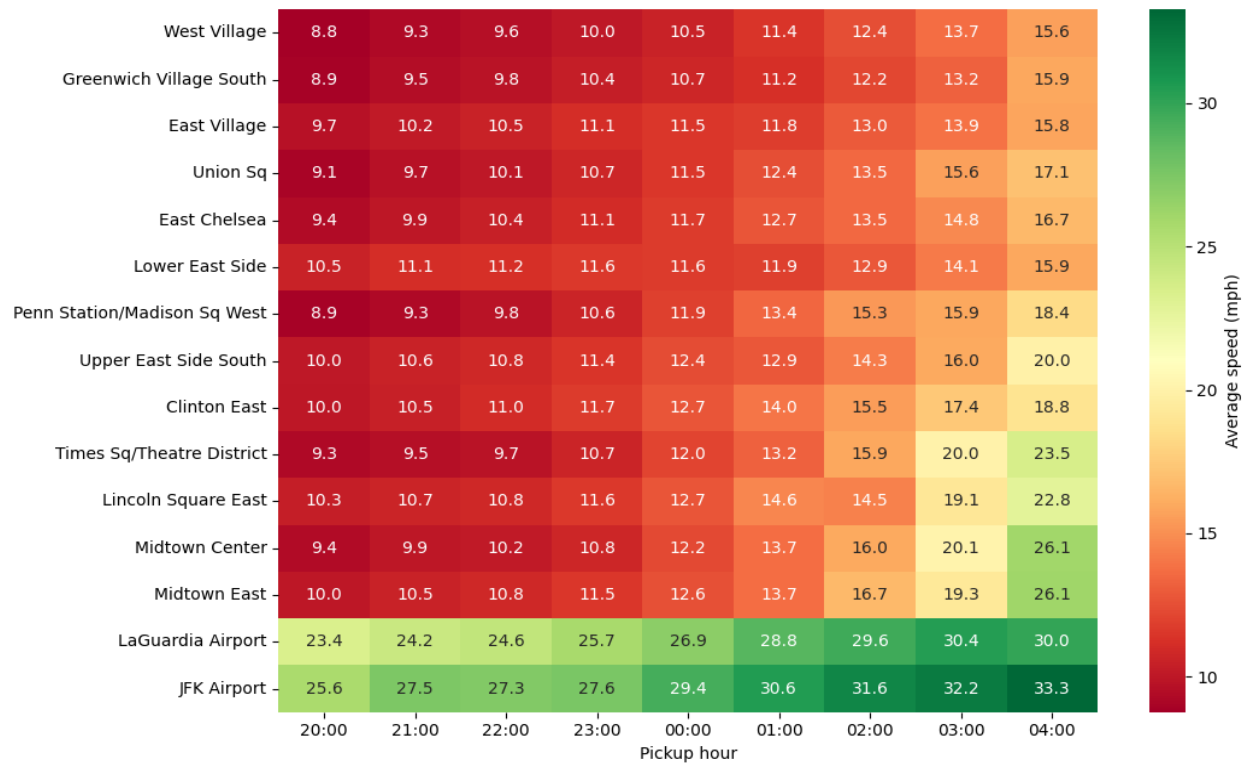


Figure 3: Average speed by zone and hour for the 15 busiest night-time pickup zones

This heatmap shows the average speed in each zone at each hour of the night. Each row is a zone and each column is an hour. Red means slow and green means fast, and the speed is printed in each box. The slowest speeds are at 20:00, when West Village is at 8.8 mph. Most zones get faster towards 04:00. However, the two airports are much faster than the other zones, at all hours. This stretches the colour scale, so the differences between the Manhattan zones look smaller than they really are. We also left out boxes with fewer than 100 trips.

#### Speed monthly trend

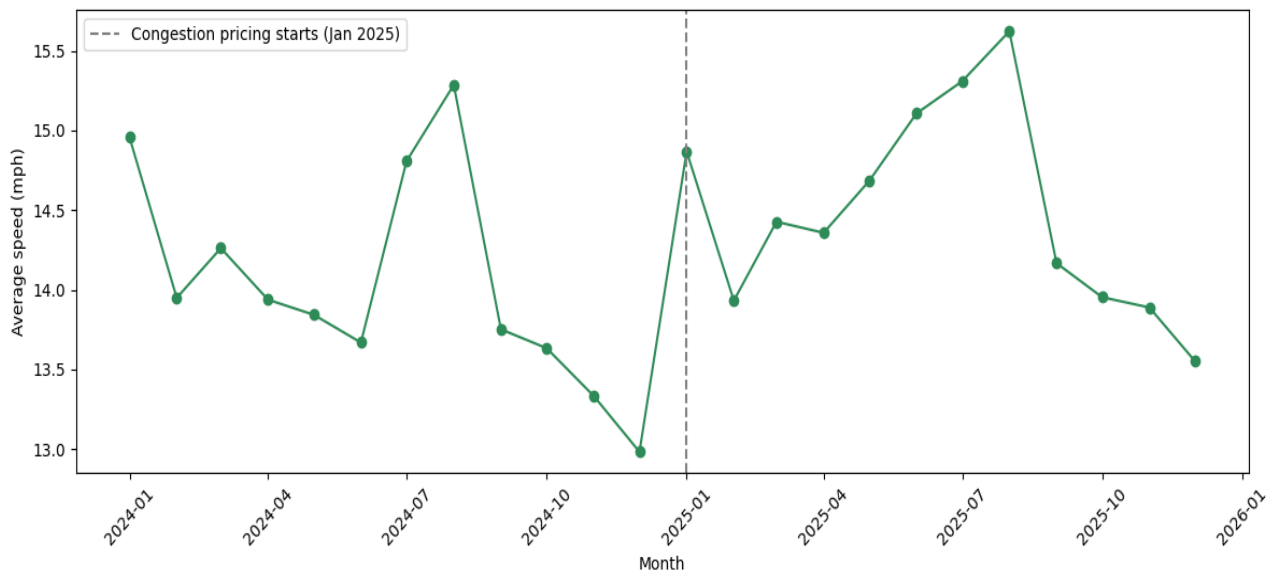


Figure 4: Average night-time taxi speed by month (2024-2025)

This line chart shows how speeds change over the two years. Speeds are highest in the summer and lowest at the end of the year. The dashed line marks the start of congestion pricing in January 2025. Although speeds are higher in 2025, the chart cannot show that congestion pricing caused this. Also, the y-axis starts at about 13 mph, so the monthly changes look bigger than they are.

## 9 Business/societal interpretation

Taxi companies can use the slow zones and hours to plan driver shifts and where to place cars. Trips that start in the Village or Penn Station area at 20:00 take much longer and they earn less per hour than the same trips at 04:00. Passengers and booking apps can use the results to give better travel times and prices in the evening. City planners and the TLC can also focus on parking rules, loading areas and traffic lights in the slowest zones. Furthermore, the faster speeds in 2025 give them a starting point to study congestion pricing. Less time in slow traffic also means less fuel use and less pollution. However, the findings have some limits. The data covers yellow taxis only, so Uber and Lyft are not included. Speed also depends on the type of trip, not only on traffic. Speeds are linked to the pickup zone because the route driven is not recorded.

## References

Apache Spark. (2025). EXPLAIN statement. Apache Spark Documentation. <https://spark.apache.org/docs/latest/sql-ref-syntax-qry-explain.html>

Apache Spark. (2025). Performance tuning. Apache Spark Documentation. <https://spark.apache.org/docs/latest/sql-performance-tuning.html>

Apache Spark. (2025). Spark SQL, DataFrames and Datasets guide. Apache Spark Documentation. <https://spark.apache.org/docs/latest/sql-programming-guide.html>

Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., & Zaharia, M. (2015). Spark SQL: Relational data processing in Spark. In Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (pp. 1383-1394). Association for Computing Machinery. <https://doi.org/10.1145/2723372.2742797>

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. Communications of the ACM, 51(1), 107-113. <https://doi.org/10.1145/1327452.1327492>

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI 2012) (pp. 15-28). USENIX Association.



```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- Project [route#122, start_zone#120L, trip_hour#118, trip_day#119, period#124, total_trip_distance#153, total_trip_duration_seconds#154L, trip_count#155L, ((total_trip_distance#153 / cast(total_trip_duration_seconds#154L as double))
* 3600.0) AS avg_speed_mph#288]
  +- HashAggregate(keys=[route#122, start_zone#120L, trip_hour#118, trip_day#119, period#124], functions=[sum(trip_distance#98), sum(trip_duration_seconds#123L), count(1)], output=[route#122, start_zone#120L, trip_hour#118,
trip_day#119, period#124, total_trip_distance#153, total_trip_duration_seconds#154L, trip_count#155L])
    +- Exchange hashpartitioning(route#122, start_zone#120L, trip_hour#118, trip_day#119, period#124, 200), ENSURE_REQUIREMENTS, [plan_id=363]
      +- HashAggregate(keys=[route#122, start_zone#120L, trip_hour#118, trip_day#119, period#124], functions=[partial_sum(trip_distance#98), partial_sum(trip_duration_seconds#123L), partial_count(1)], output=[route#122,
start_zone#120L, trip_hour#118, trip_day#119, period#124, sum#199, sum#200L, count#201L])
        +- Project [trip_distance#98, (unix_timestamp(tpep_dropoff_datetime#96, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true) - unix_timestamp(tpep_pickup_datetime#95, yyyy-MM-dd HH:mm:ss, Some(Etc/UTC), true)) AS
trip_duration_seconds#123L, hour(tpep_pickup_datetime#95, Some(Etc/UTC)) AS trip_hour#118, dayofmonth(cast(tpep_pickup_datetime#95 as date)) AS trip_day#119, PULocationID#101L AS start_zone#120L, concat_ws(-,
cast(PULocationID#101L as string), cast(DOLocationID#102L as string)) AS route#122, concat_ws(, cast(year(cast(tpep_pickup_datetime#95 as date)) as string), lpad(cast(month(cast(tpep_pickup_datetime#95 as date)) as string), 2, 0)) AS
period#124]
          +- Filter ((year(cast(tpep_pickup_datetime#95 as date)) = 2025) OR (year(cast(tpep_pickup_datetime#95 as date)) = 2024))
            +- FileScan parquet [tpep_pickup_datetime#95,tpep_dropoff_datetime#96,trip_distance#98,PULocationID#101L,DOLocationID#102L] Batched: true, DataFilters: (((year(cast(tpep_pickup_datetime#95 as date)) = 2025) OR
(year(cast(tpep_pickup_datetime#95 as d..., Format: Parquet, Location: InMemoryFileIndex(24 paths)[file:/content/drive/MyDrive/mit805/parquet_files_2024_2025/yellow_tri..., PartitionFilters: [], PushedFilters: [], ReadSchema:
struct<tpep_pickup_datetime:timestamp_ntz,tpep_dropoff_datetime:timestamp_ntz,trip_distance:doubl...
```

Results

Slowest zones

| Zone                          | Borough   | trips   | avg_speed_mph      |
|-------------------------------|-----------|---------|--------------------|
| Greenwich Village North       | Manhattan | 450046  | 10.281608701040856 |
| Penn Station/Madison Sq West  | Manhattan | 747003  | 10.327819460174393 |
| West Village                  | Manhattan | 1107072 | 10.337682326310556 |
| Union Sq                      | Manhattan | 799333  | 10.380676250566689 |
| Midtown Center                | Manhattan | 1046036 | 10.412650378067093 |
| Greenwich Village South       | Manhattan | 787577  | 10.50121913063746  |
| Central Park                  | Manhattan | 192314  | 10.529193583950386 |
| Meatpacking/West Village West | Manhattan | 431051  | 10.572553472874036 |
| SoHo                          | Manhattan | 289668  | 10.607790285251397 |
| Midtown North                 | Manhattan | 638029  | 10.631572268785371 |

| TRIP_HOUR | TRIPS   | AVG_SPEED_MPH |
|-----------|---------|---------------|
| 20:00     | 5088848 | 13.009072     |
| 21:00     | 5211695 | 13.353215     |
| 22:00     | 4866726 | 13.736902     |
| 23:00     | 3806629 | 14.943463     |
| 00:00     | 2671339 | 15.608786     |
| 01:00     | 1741900 | 15.207083     |
| 02:00     | 1138797 | 15.248767     |
| 03:00     | 762337  | 16.652821     |
| 04:00     | 581475  | 19.632811     |

Trips analysed: 25,869,746  
Overall night speed: 14.2 mph  
Slowest hour: 20:00 (13.0 mph)  
Fastest hour: 04:00 (19.6 mph)

